TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS:

Tyler Jan Panchana Cevallos

PARALELO:

"A"

SEMESTRE:

Quinto semestre

ACTIVIDAD:

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad:

Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

Actividad # 1

Análisis del problema

Identificar los actores principales.

Actores principales	**Rol en sistema**
Reportante/Ciudadano	Reporta un animal en situación vulnerable, que necesite rescate, y crea el caso inicial. Consulta el estado del caso.
Voluntario/Veterinario	Atiende los casos: actualiza estados, registra tratamientos y seguimientos del animal.
Administrador del refugio	Gestiona el sistema completo: asigna personal, aprueba adopciones, administra usuarios y casos.
Actores secundarios	**Rol en sistema**
Adoptante	Persona interesada en adoptar. Consulta animales disponibles para adopción, presenta solicitud y formaliza la adopción.
Donante/Colaborador	Persona u organización que apoya al refugio. Registra donaciones (económicas, insumos) y puede recibir constancia o seguimiento de su aporte.

Requerimientos Funcionales (RF)

RF1: Crear un caso de rescate con los datos del animal y del reportante.

RF2: Categorizar la especie del animal (perro, gato, fauna silvestre).

RF3: Asignar un voluntario o veterinario encargado a cada caso.

RF4: Gestionar los estados del proceso (Rescatado, En tratamiento y Adopción).

RF5: Notificar automáticamente a los voluntarios ante nuevos rescates.

RF6: Registrar el historial de cada animal (tratamientos, cambios de estado).

RF7: Registrar solicitudes de adopción y gestionar su aprobación.

RF8: Registrar y consultar donaciones al refugio.

Requerimientos no funcionales (RNF)

Escalabilidad: el sistema debe soportar un crecimiento en el número de casos, refugios y usuarios sin degradar el rendimiento.

Mantenibilidad: el código debe estar organizado siguiendo principios SOLID, facilitando su modificación y extensión futura.

Trazabilidad de versiones: el desarrollo debe gestionarse con Gitflow, permitiendo rastrear cada cambio y mantener ramas estables.

2. Crear un diagrama de casos de uso para entender las interacciones del personal con la plataforma.

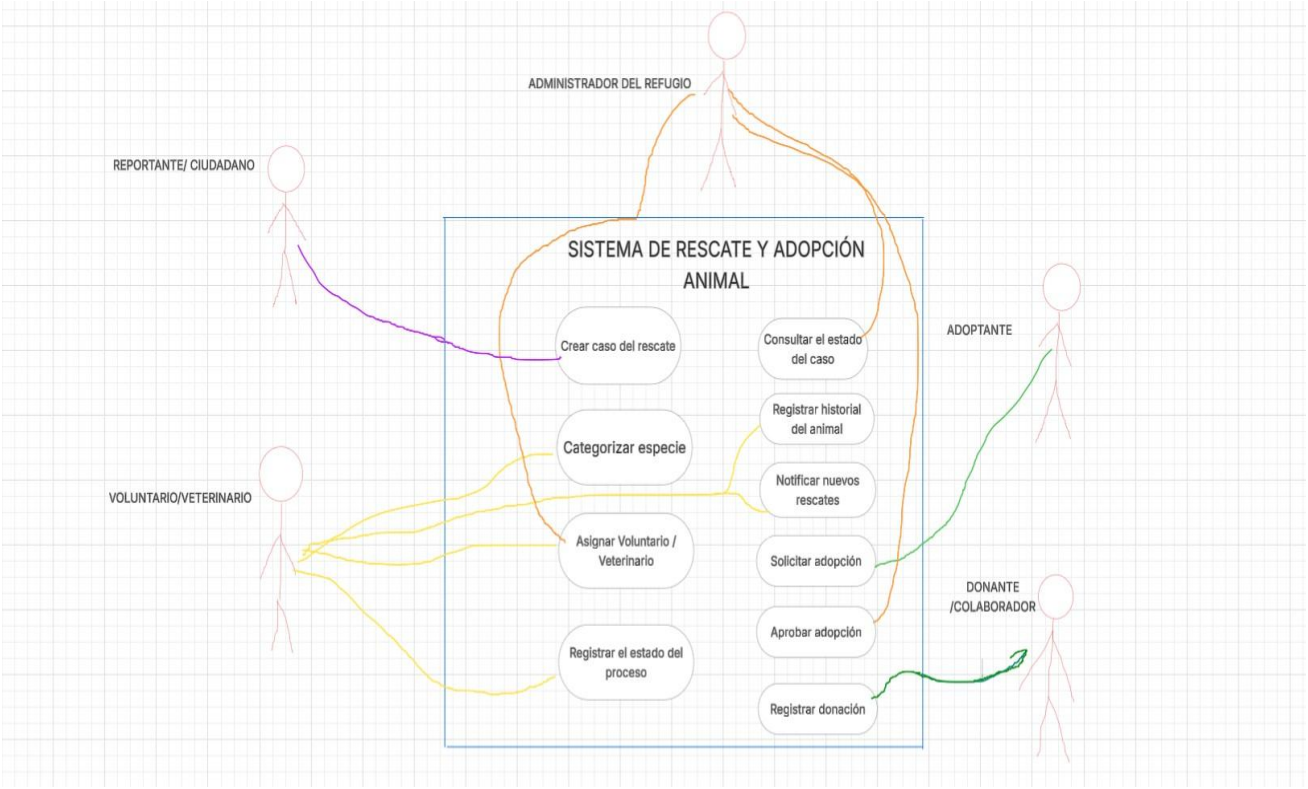
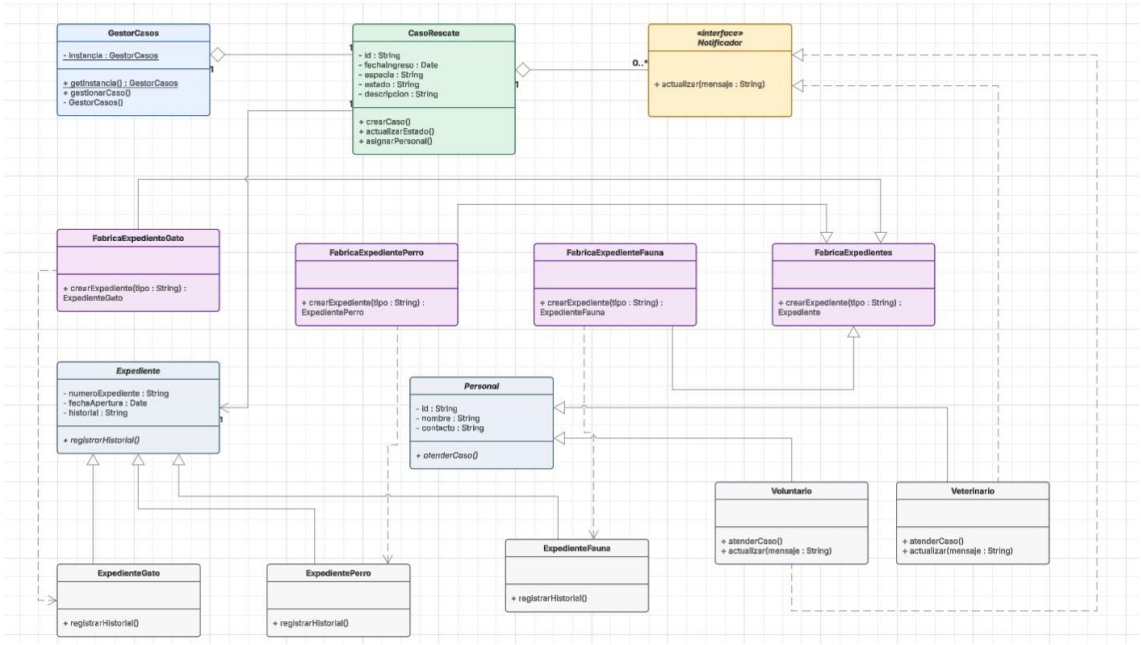


Diagrama UML



Singleton — GestorCasos: garantiza una única fuente de verdad para los casos registrados. Sin él, cada módulo tendría su propia lista y se perdería la consistencia. Alternativa descartada: variables globales (difíciles de testear y de controlar).

Factory Method — FabricaExpedientes: el sistema no sabe de antemano si el rescate es de gato, perro o fauna. La fábrica encapsula esa decisión. Alternativa descartada: un switch/if sobre el tipo dentro del cliente, que obligaría a modificar código cada vez que se agregue una especie nueva (viola abierto/cerrado).

Observer — Notificador: cuando cambia el estado de un CasoRescate, hay que avisar a voluntarios y veterinarios sin que el caso los conozca en concreto. Alternativa descartada: que CasoRescate llame directamente a cada tipo de personal, lo que lo acoplaría a las subclases.

Evidencias

Enlace del Git

https://github.com/tpanchana6133-max/Rescate-adopci-n-animal-.git

Enlace de LucidChart

Diagrama de uso de casos.

https://lucid.app/lucidchart/59e2ac36-c91e-45ed-9319-73c72b716b83/edit?viewport_loc=-2647%2C-3660%2C17294%2C7318%2C0_0&invitationId=inv_27fa2742-e684-4177-b7d9-2737b32ef4de

Enlace de LucidChart

Diagrama UML

https://lucid.app/lucidchart/f9a0c582-d20e-4af1-9ca9-890dc508e78b/edit?viewport_loc=-1429%2C-390%2C5031%2C2129%2C0_0&invitationId=inv_2c124000-a7f3-4d49-bc60-55ce78fef7be

Capturas de ejecución del Git

Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)